

Firstname : Xavier

Lastname : KNOEPFFLER-POUT

Firstname :Muhammad Omais

Lastname :RANA

# Practical Session – Proxmox v4.00<sub>iot</sub>

## 1. Context

Proxmox is a hypervisor, i.e., a minimalist operating system that allows the creation and management of Virtual Machines (VMs).

Services used by end users (like web server, mail server, or specific applications) are deployed inside these VMs. This design provides:

- Better isolation between applications
- Easier backup and restore procedures
- Simplified migration of workloads

In this practical work that will span over multiple sessions, you will explore the **High Availability (HA)** feature of Proxmox, which requires at least **three Proxmox nodes**. You will also set up redundant storage servers to host the application VMs.

---

Since Proxmox requires a **clean installation** (the hard drive must be wiped), and it is not possible to provide 3 to 5 physical machines per student, we will adopt a **virtualized approach**:

- Proxmox will be installed **inside a provided Proxmox server** (yes, Proxmox inside Proxmox).
- Proxmox's **virtual networking features** will be used to interconnect these virtual machines.
- The resulting setup will emulate a small **datacenter environment**, with several nodes and multiple network interfaces.
- Obviously, while this nested architecture is really convenient and allow student to fully manage Proxmox hosts, it is not meant to be used in a real production environment. In a real deployment, Proxmox hosts would be real computers !

---

During this first session, you will:

1. Connect to your Proxmox host (one host per group of two students).
2. Create 2 LXC container and learn the difference between privileged and non-privileged
3. Create a new VM and configure its resources (CPU, RAM, disk).
4. Install Proxmox insode this VM from an ISO image

➡ In subsequent sessions, you will gradually create additional VMs to build a **Proxmox cluster with HA and shared storage**.

---

## ⚠ Important Notes

- **Networking:** Proper configuration of network interfaces will be crucial for cluster communication.

# Connecting to your server

## Address and ports

A Proxmox server is provided to each group of students. This server is **accessible only from the university's wired network**. If you want to access it from your laptop (even with eduroam) or from your home, you will need to use the VPN.

Management of Proxmox is usually done through its web interface, but you may also need to connect with ssh.

- Web interface on a Proxmox server is always HTTPS on port 8006
- SSH is configured on port 22

However, to prevent problems with the University's network, your Proxmox server is not accessible directly. It sits inside a DMZ, behind a firewall. To access your server, you will need to connect to this firewall on specific ports. Those ports are forwarded to your server.

Hostname / IP address of the firewall: `boba.nfc.univ-fcomte.fr` / `172.20.43.9`

The template for the port redirection is the following: 9xxy

xx: group number (from 01 to 09)

y: range (0 to 9)

port y = 0 is forwarded to your server on port 22 (for ssh)

port y = 1 is forwarded to your server on port 8006 (for Proxmox's WebUI)

port y = 2..9 are forwarded to your server on ports 8007 to 8014 (you will need them later)

### Examples:

Your are group number 03

To connect to the WebUI of your server: <https://boba.nfc.univ-fcomte.fr:9031>

To connect using ssh to your server: `boba.nfc.univ-fcomte.fr:9030`

Your are group number 07

To connect to the WebUI of your server: <https://boba.nfc.univ-fcomte.fr:9071>

To connect using ssh to your server: `boba.nfc.univ-fcomte.fr:9070`

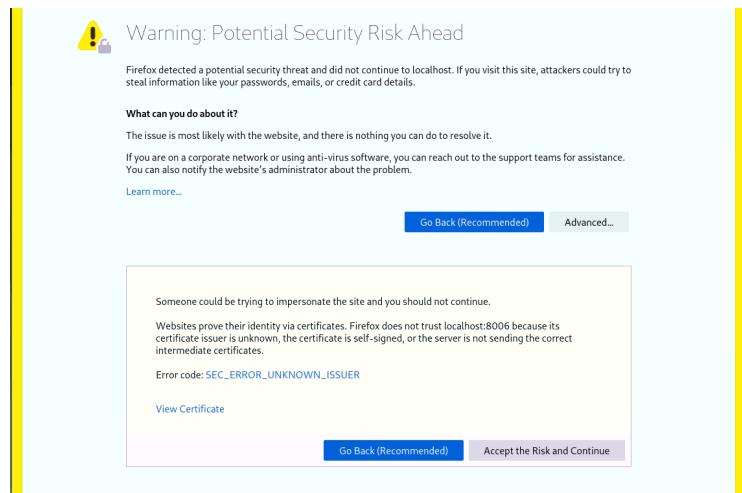
## Login and WebUI

On your classroom computer, what URL should you use to access the web interface of your proxmox server ?

[Answer here](#)

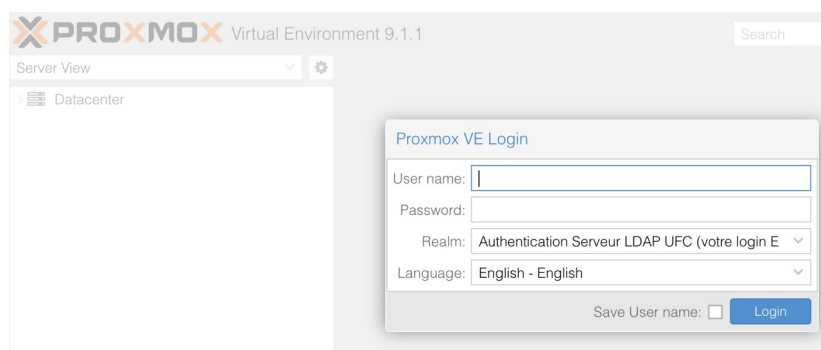
As we are group 2, we will be using | <https://boba.nfc.univ-fcomte.fr:9021/> |

⚠ **Important:** The browser will display a warning about a self-signed certificate.



- This is normal.
- Accept the warning and proceed to the web interface.

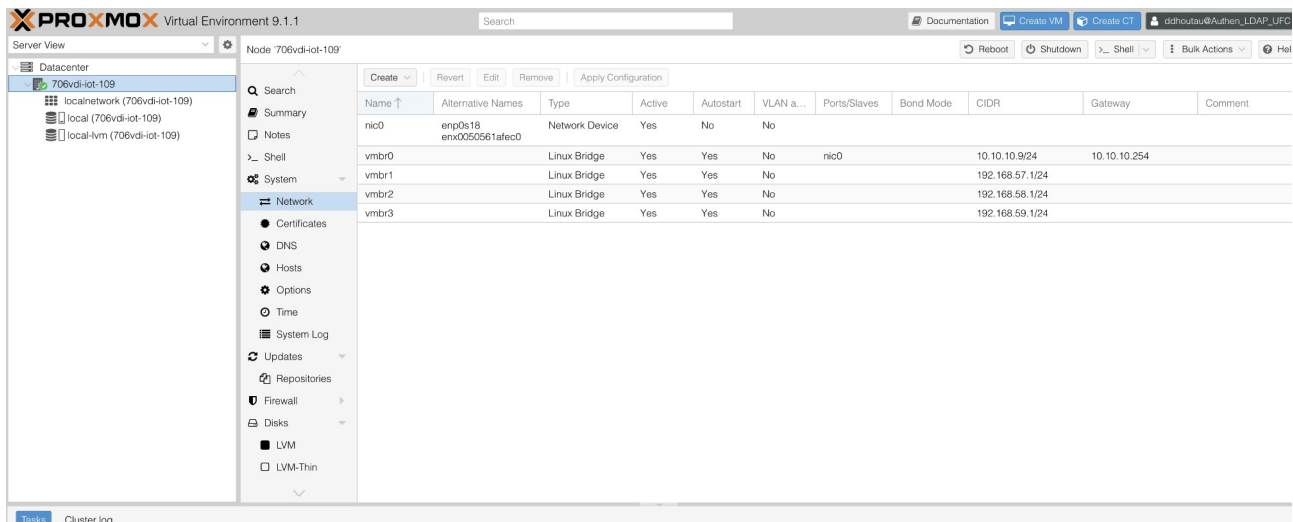
You should get the login screen:



- Use your usual **university's login and password.**
- Make sure to select the correct **realm: "Serveur LDAP UFC"**

## Navigating the Management Interface

Once logged in, you will see the **Proxmox management screen.**



- **View selector (1):** choose the interface view.
  - For now, stay in **Server view**.
- **Server list (2):** shows all servers you can currently manage.
  - You should only see **706vdi-iot-xxx**.
- **VM list:** below each server, you can see all **existing virtual machines** and the **available storage**.
  - Initially, there should be **no Vms**.
  - There should be two storages: local and local-lvm

## Creating Virtual Machines

- To create new virtual machines, use the buttons at the **top right**:
  - If your browser's window is too narrow, you may not see those buttons !
  - **Create VM** → for full virtual machines
  - **Create CT** → for containers

These buttons allow you to start the standard Proxmox VM creation wizard.

## Network configuration of your Proxmox host

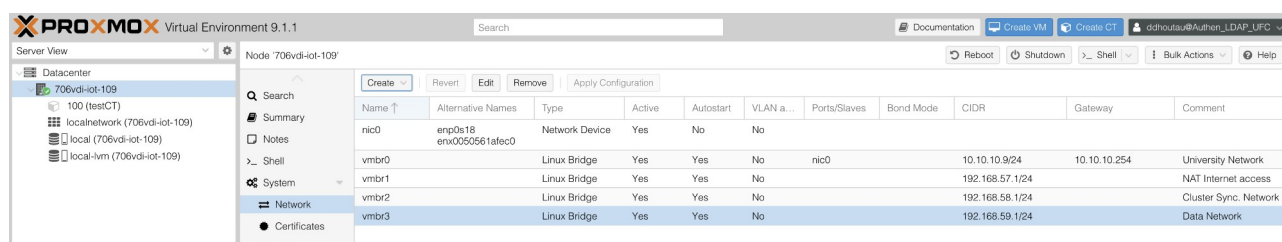
Your host has multiple network interfaces:

- nic0/enp0s18 is the physical network interface, connected to the university's network  
It's IP address should be in the 10.10.10.0/24 range (10.10.10.7 for example)
- vmbr0 to vmbr3 are virtual bridges that are connected to nothing yet
  - vmbr0 (10.10.10.xxx) is binded to the physical interface and carry its IP address (visible on the university network)

- vmbr1 (192.168.57.1) will be configured to serve as a bridge to the internet for all your VM and contains. You will have to configure NAT on it (explained later)
- vmbr2 (192.168.58.1) will be the bridge to the synchronization network (when you will have a cluster of multiple Proxmox servers)
- vmbr3 (192.168.59.1) will be the bridge to the data network (for redundant network storage)

To help preventing configuration mistakes later on, we will add short descriptions to the virtual bridges in the network configuration screen.

Select your host on the left panel, then System / Network in the menu. You should see all interfaces.



Double-click on each virtual bridge (vmbr0 .. vmbr4) and add the following descriptions in the “comment” field:

|       |                       |
|-------|-----------------------|
| Vmbr0 | University Network    |
| Vmbr1 | NAT Internet Access   |
| Vmbr2 | Cluster Sync. Network |
| Vmbr3 | Data Network          |

Apply the changes with the “Apply Configuration” button on the network screen.

## Nat configuration for vmbr0

We use NAT in our setup to isolate the VMs created by each group of students. The VMs of a group should see each other on the same network, but should not see the VMs from another group.

You have to activate port forwarding on your Proxmox host, and this cannot be done from the WebUI.

You need to either

- Open a shell (select your host in the server list, then “shell” in the menu)
- Connect to your server using ssh from your own computer (remember that you have to connect to a specific port of the “boba” firewall. At this point connecting with ssh may not work (it requires an action from the university technical staff)
- ... if both options are unavailable, ask the teacher !

Adding a rule in /etc/sysctl.d will tell Proxmox to activate it at boot time. Edit the file /etc/sysctl.d/99-proxmox-nat.conf with nano

```
bob@706vdi-iot-111:~$ sudo nano /etc/sysctl.d/99-proxmox-nat.conf
```

Write the following content inside: `net.ipv4.ip_forward=1`

CTRL-O to save

CTRL-X to exit nano

Then immediately activate this rule:

```
bob@706vdi-iot-111:~$ sudo sysctl -p /etc/sysctl.d/99-proxmox-nat.conf
```

Make sure the rule was applied. The following command should return “1”

```
bob@706vdi-iot-111:~$ sudo sysctl net.ipv4.ip_forward
```

The kernel is now able to do IP forwarding, but we need to add rules to tell it exactly what we want. This is done with tools like nftables (newer) or iptables (older, but still supported).

This first rule will do the actual translation, allowing Vms to access the internet by using the IP address of the host.

```
bob@706vdi-iot-111:~$ sudo iptables -t nat -A POSTROUTING -s 192.168.57.0/24 -o vmbr0 -j MASQUERADE
```

What is the exact meaning of this rule ?

[Answer here](#)

First of all, iptables is a command-line utility which configures the built-in Linux kernel firewall, it allows set administrators to define chained rules that control incoming and outgoing network traffic.

With the use of -t nat, we are telling the command to use the nat table

The -A options leads to Appending a rule to a chain

The POSTROUTING part Alters packets after they are routed. This alteration happens after a decision.

MASQUERADE alters packet's source addresses for dynamically assigned Ips, we are masking it

So basically, a way to explain what the command does is, by using iptables, any traffic coming from the internal network (the 192.168.57.0/24 address) that is heading outside with vmbr0, mask the private IP address with our own IP address.

We need two more rules to let the packets go from the VMs to the outside (first rule) and from the outside to the VMs, but only if they are a response or related to an existing flow (second rule). Those rules allow communication with the outside but prevent unsolicited connexion to our VMs.

```
bob@706vdi-iot-111:~$ sudo iptables -A FORWARD -i vmbr1 -o vmbr0 -j ACCEPT

bob@706vdi-iot-111:~$ sudo iptables -A FORWARD -i vmbr0 -o vmbr1 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

To make the modification permanent, we can add those 3 rules to `/etc/network/interfaces`. Use nano to edit this file:

```
bob@706vdi-iot-111:~$ sudo nano /etc/network/interfaces
```

and change the vmbr0 section as follow:

```
auto vmbr0
iface vmbr0 inet static
    address 10.10.10.9/24
    gateway 10.10.10.1
    bridge-ports nic0
    bridge-stp off
    bridge-fd 0
```

```
post-up iptables -t nat -A POSTROUTING -s 192.168.57.0/24 -o vmbr0 -j MASQUERADE
post-up iptables -A FORWARD -i vmbr1 -o vmbr0 -j ACCEPT
post-up iptables -A FORWARD -i vmbr0 -o vmbr1 -m state --state ESTABLISHED,RELATED
-j ACCEPT
post-down iptables -t nat -D POSTROUTING -s 192.168.57.0/24 -o vmbr0 -j MASQUERADE
post-down iptables -D FORWARD -i vmbr1 -o vmbr0 -j ACCEPT
post-down iptables -D FORWARD -i vmbr0 -o vmbr1 -m state --state
ESTABLISHED,RELATED -j ACCEPT
#University network
```

## 6. Key Concepts of Proxmox

### 6.1 Context and Advice

In a real job, you will frequently need to **search for and read documentation independently**.

⚠ Be aware of the following:

- Always check the **version** of the software you are reading about. Older tutorials or guides may describe steps that no longer work.
- Many online tutorials are **outdated**. Cross-check any instructions before applying them.
- AI tools such as ChatGPT can give helpful answers, but sometimes include **completely incorrect information**. Always verify with trusted sources.

There is **no shortcut**: verifying information with official or trusted sources is mandatory.

What will be your primary source of information for Proxmox? Write the URL of the website you will use as your main reference.

[Answer here](#)

Most likely the official Documentation which comes with the actual Proxmox site, the versions is stated as v9.1.1 and was last updated in November 19<sup>th</sup> 2025

We can also look up online for help, however we will only look up help if it is actually in line with our respective version, 9.1.1 or 9.1.0

What is proxmox ? Is it a software you install in your operating system ? Is it an operating system ? Is it written from scratch or is it based on commonly used software ? (if so, what software(s) ?)

[Answer here](#)

Proxmox is a Virtual Environment which is an Operating System which is heavily based on Debian Linux.

## 7. Storage Space

### 7.1 Context

Proxmox supports a wide variety of **filesystems, partition schemes, and networked storage volumes**.

Examples include:

- Local directory
- LVM volume
- CIFS/SMB share
- iSCSI volume
- Distributed storage (Ceph, etc.)

Each storage can be mounted in Proxmox and configured to allow specific types of content.

### 👉 Examples:

- One storage can be restricted to **backups only**.
- Another may be reserved for **virtual machine disk images**.
- Some combinations are not possible (e.g., storing ISO files directly on an LVM volume).

## 7.2 Task

Your first task is to **list all the storage spaces available** to you in your Proxmox server.

For each storage, indicate:

1. **Name**
2. **Type**
3. **Content allowed**
4. **Remaining space**

- Use the **Proxmox graphical interface** (web UI) to explore the storage configuration.
- Check the **Datacenter** or **Server view** to find storage definitions.
- Compare the differences between local storage and shared/distributed storage (if any).

List all the storage spaces available in your Proxmox setup, with their type, their allowed content and their free space.

[Answer here](#)

Here are the different storage spaces available in our Proxmox setup:

We mainly have 1 main device which is split into 3 different partitions:

/dev/sda | It is a QEMU\_HARDDISK and is connected to the serial drive-scsi0  
    /dev/sda1 | its type is a partition and works as the BIOS boot | it has 1.03MB of space left  
    /dev/sda2 | its type is a partition and works as an EFI | it has 1.07GB of space left  
    /dev/sda3 | its type is a partition and works as an LVM | It has 213.67GB of space left | In this volume, we won't be able to add in any ISO files due to its LVM nature

When creating new virtual machines, where will you store them ?

[Answer here](#)

Logically, we will be storing them in /dev/sda3, its the only one which would have enough space to hold them

# 8. The Two Different Types of Virtual Machines

## 8.1 Context

Proxmox supports two very different virtualization technologies:

- **KVM (Kernel Virtual Machine):** full virtualization
- **LXC (Linux Container):** lightweight container-based virtualization

These two types are designed for different use cases, each with its **strengths and limitations**.

---

## 8.2 Task

Using the **Proxmox documentation (Features section)**, research and describe:

1. **What makes them different**
2. **What you can install inside each**
3. **How they compare regarding resource usage** (RAM, CPU, disk, etc.)

Main advantages and limitations of KVM Virtual Machines:

[Answer here](#)

Advantages:

- KVM has full isolation since each VM runs its own Kernel.
- Proxmox emulates hardware components using QEMU.
- Support for specialized kernels and custom drivers.
- Can run virtually any x86 operating system, including Windows.

Limitations:

- Requires more RAM and CPU cycles because it must run a full kernel and emulate hardware components.
- Each VM requires a complete OS installation and a dedicated virtual disk image.
- Must go through a virtual BIOS/UEFI and a complete OS boot sequence, taking longer.

Main advantages and limitations of LXC Virtual Machines:

[Answer here](#)

Advantages:

- Runs at near-native speed because there is no hardware emulation layer.
- Extremely low memory footprint since a container only consumes the RAM needed by its active processes.
- Allows you to run many more containers than VMs on the same hardware due to the lack of overhead.

Limitations:

- You can only run Linux-based distributions.
- All containers share the host's Linux kernel. This means if the host kernel crashes, all containers go down.
- You cannot install a different kernel or specific kernel modules inside a container

## 9. Configuring the Repositories and Updating the System

### 9.1 Context

When logging into Proxmox, you may have noticed a warning about **not having a subscription license**.

- This is **completely legal**.
- However, it affects the **repositories** (software sources) that can be used for updates.

By default, Proxmox is configured to use the **enterprise repositories**, which are only accessible with a paid license.

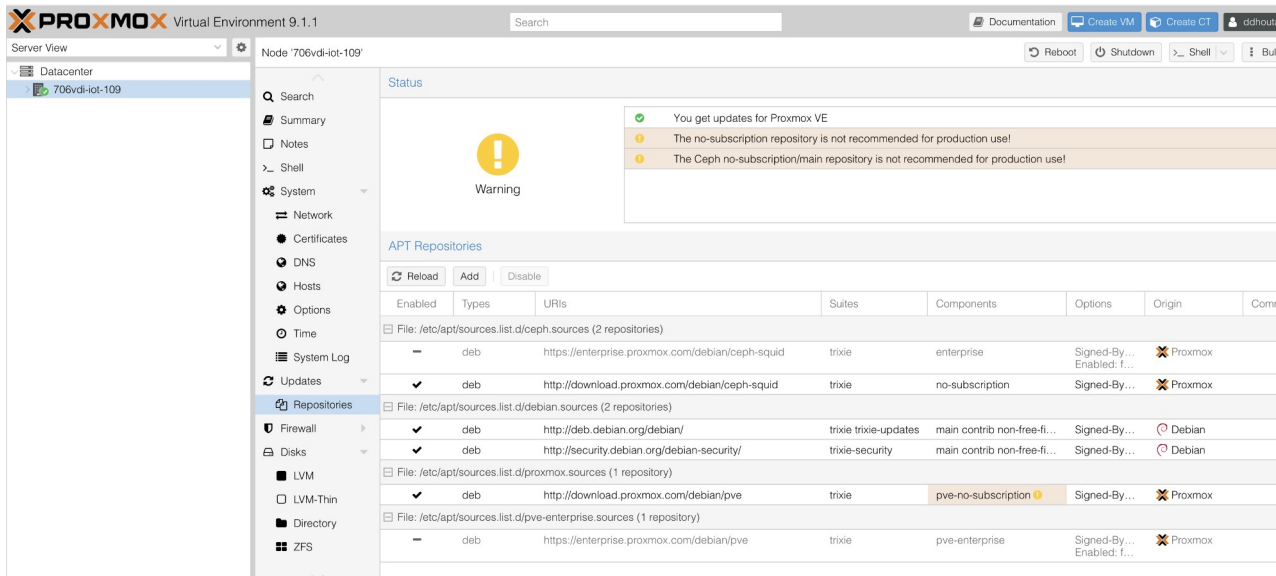
- These repositories contain packages that are **extensively tested** by the Proxmox team.
- Without a license, commands like `apt update` will fail due to authentication errors.

👉 To work without a subscription, we must configure the **no-subscription repositories** instead.

### 9.2 Task

1. Open the **Proxmox repository configuration** on your VM. Note: you have two ways to configure the repositories:
  1. The command line and manual edition of apt's configurations files.

## 2. Using the WebGUI



2. Follow the official documentation to enable the **no-subscription repositories**:  
[Proxmox VE Package Repositories](#)
3. Disable or remove the enterprise repositories (otherwise apt update will fail).
4. Once configured, update the package lists with:  

```
sudo apt update
```

# 10. Using LXC Containers

## 10.1 Context

Now that your system is **up-to-date**, you can start experimenting with **containers**.

In Proxmox, before creating a container, you need a **template**.

For this exercise, you will use the **debian-13-standard** template, which is provided directly by Proxmox.

## 10.2 Downloading the Template

1. In the web interface, select the **storage** where you want to store the template.
  - ⚠ Remember: Proxmox only allows specific content types in each storage.
  - Make sure the storage supports **CT Templates**.
2. In the left panel, click on **CT Templates**.
3. On the right panel, open the **Templates** tab.
4. Download the **debian-13-standard** template.
  - Size: ~120 MB

- Download should take less than a minute.

## 10.3 Creating the Container

Once the template is available, you can create a new LXC container with the following parameters:

- **General**
  - Name: `testCT1`
  - Check **Unprivileged container**
  - Root password: `tpiot` (enter twice)
- **Template**
  - Source: the file you just downloaded
- **Root Disk**
  - Storage: (only one available)
  - Size: 2 GiB
- **CPU**
  - 1 core
- **Memory**
  - 512 MiB
- **Network**
  - Bridge: `vbr1` (NAT internet access)
  - IPv4 address: `192.168.57.2/24`
  - Gateway: `192.168.57.1`
  - IPv6: None
- **DNS**
  - Use host parameters

⚡ Once you validate, the container will be created from the template.

- Expected duration: ~30 seconds

# 11. Bridges and Network Configuration

## 11.1 Context

You will now test the **network configuration** of your first container.

Start your container **testCT1** and open a **console**.

- Login using the account:

- **Username:** root
- **Password:** tpiot

⚡ Notice how quickly the container starts — usually just a few seconds!

## 11.2 Internet Connectivity Test

Inside the container console, check that it can connect to the internet, for example with:

```
testCT1# ping debian.org
```

If you receive replies, the container is correctly using the `vmbr0` bridge.

## 11.3 Adding a Second Interface

We will now attach a second network interface to the container.

1. In the **Proxmox web interface**, select the container **testCT1**.
2. Go to the **Network** section.
3. Click **Add** and configure:
  - **Name:** eth1
  - **Bridge:** vmbr2
  - **IPv4 address:** 192.168.58.2/24
  - **Gateway:** none
  - **IPv6:** none

⚡ Once validated, the new interface will appear immediately inside the container (no reboot required).

---

## 11.4 Verifying Interfaces

Inside the container, check the available interfaces and routes with

```
testCT1# ip a
```

```
testCT1# ip r
```

Put here a screenshot of the result, and explain what is the default route

[Answer here](#)

```

root@testCT1:~# ip r
default via 192.168.57.1 dev eth0 proto kernel onlink
192.168.57.0/24 dev eth0 proto kernel scope link src 192.168.57.2
192.168.58.0/24 dev eth1 proto kernel scope link src 192.168.58.2
root@testCT1:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:f3:ac:4a brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.57.2/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fef3:ac4a/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
8: eth1@if9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:a8:e3:3d brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.58.2/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fea8:e33d/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@testCT1:~# █

```

## 12. A Second Container

### 12.1 Context

To test communication between containers, you will now create a **second container**. This will allow you to verify connectivity through the **vmbr1 private bridge**.

### 12.2 Creating testCT2

Create a new container with the same procedure as **testCT1**, except for the following differences:

- **General**
    - Name: testCT2
    - ⚠ Uncheck **Unprivileged container** (this container must be **privileged**)
  - **Network**
    - eth0: via vmbr1 (NAT internet access)
      - IPv4 address: 192.168.57.3/24
      - gateway: 192.168.57.1
    - eth1: via vmbr1
      - IPv4 address: 192.168.58.3/24
      - gateway: none
-

## 12.3 Verifying Connectivity

Once both containers are started:

1. From `testCT1`, ping `testCT2` on its private address:

```
testCT1# ping 192.168.58.2
```

2. From `testCT2`, ping `testCT1` on its private address:

```
testCT2# ping 192.168.58.3
```

3. From either container, ping the **Proxmox host** on the same network (`192.168.58.1`):

```
testCT1# ping 192.168.58.1
```

If all pings succeed, your private network (`vmbr2`) is correctly configured.

# 13. Processes Inside Non-Privileged Containers

## 13.1 Context

In this section, you will explore how **processes** are managed inside an **unprivileged container** (`testCT1`) compared to the Proxmox host (`706vdi-iot-xxx`).

You will observe the difference in process isolation and memory usage.

## 13.2 Tasks

### Step 1: Start a Program Inside `testCT1`

1. Open a console on **`testCT1`**.
2. Start a simple program, for example:

```
testCT1# less /etc/passwd
```

- This opens a text viewer.
3. Suspend the program with:
    - Press **CTRL+Z**
    - You should return to the command line, while `less` remains in memory (paused).

### Step 2: List Processes Inside the Container

1. Display all processes inside `testCT1`:

```
testCT1# ps aux
```

2. Observe the list:

- You should find the **less** process (note down its **PID** and **User**).

**PID: 368 | User: root**

- You should also see around 15 other processes, including:
  - `init` (always PID 1 inside the container)
  - daemons (e.g., for login and shell)

### Step 3: Check Memory Usage Inside the Container

1. Display memory information:

```
testCT1# free -h
```

2. Expected result:

- Total RAM available  $\approx$  512 MiB (container allocation)
- Used RAM  $\approx$  25–30 MiB

### Step 4: Compare With the Proxmox Host

1. Open a shell console on `706vdi-iot-xxx` (the Proxmox server).

2. List processes:

```
706vdi-iot-xxx$ ps aux
```

- The list should be **much longer** than inside the container.

3. Display memory usage:

```
706vdi-iot-xxx$ free -h
```

- Total RAM available: 6 GiB (server allocation)
- Used RAM: several GiB

4. Search for specific processes:

```
706vdi-iot-xxx$ ps aux | grep init  
706vdi-iot-xxx$ ps aux | grep less
```

- Check whether the `init` and `less` processes of the container appear in the **host's process list**.

Explain what you observe and give your interpretation:

[Answer here](#)

Our main interpretation, would be that the difference of info given to us with is more condense and easier to digest, which would in return, make the actual virtual machines faster as, compared to the main host: The host uses a lot more memory than the virtual machine and has a lot more processes going on; which in return would allow us to analyze data way more easily

## 14. Processes Inside Privileged Containers

### 14.1 Context

In the previous section, you observed how processes from an **unprivileged container** appear from both inside the container and from the Proxmox host.

Now you will repeat the same experiment with the **privileged container** (`testCT2`) to compare behaviors.

### 14.2 Tasks

#### Step 1: Start a Process Inside `testCT2`

1. Open a shell console on **testCT2**.
2. Start a program, for example:

```
testCT2# less /etc/passwd
```

3. Suspend it (keep it in memory but paused):
    - Press **CTRL+Z**
-

## Step 2: Observe From the Host ( 706vdi-iot-xxx )

### Return to a shell on the Proxmox host.

1. List all running processes, but filter only for `less`:

```
706vdi-iot-xxx$ ps aux | grep init
706vdi-iot-xxx$ ps aux | grep less
```

What do you observe ? Look at the name or user id (it's the first column displayed by "ps aux"). How do you interpret this difference ?  
What are the differences between a privileged and unprivileged container ? (you also look in the documentation !)

[Answer here](#)

**The privileged container has the mapped id of the host while the unprivileged container has the mapped id of an unprivileged user.**

## Installing your first Proxmox virtual server.

You are going to create the first virtual Proxmox of your cluster.

You will first have to **get the ISO image** for the last version of Proxmox (9.1 at the time of writing of this document). You can either:

- Download it on your computer using your web browser, and then upload it on the correct storage space on your proxmox server.  
When you select a storage that can contain ISO images, an "ISO Images" menu entry with an "upload" button should appear.
- Upload it directly on the correct storage space using the URL and Proxmox's WebUI.  
Use the "Download from URL" to go this route.

From what you already learned from this course, you should be able to determine which kind of virtual machine (KVM or container) should be used to install a Proxmox server inside from an ISO image

Type of virtual machine required for a nested Proxmox server ?

Answer here

A KVM (virtual machine) is required because LXC is not capable of acting as a hypervisor for a nested setup.

Use the corresponding button (“Create VM” or “Create CT”).

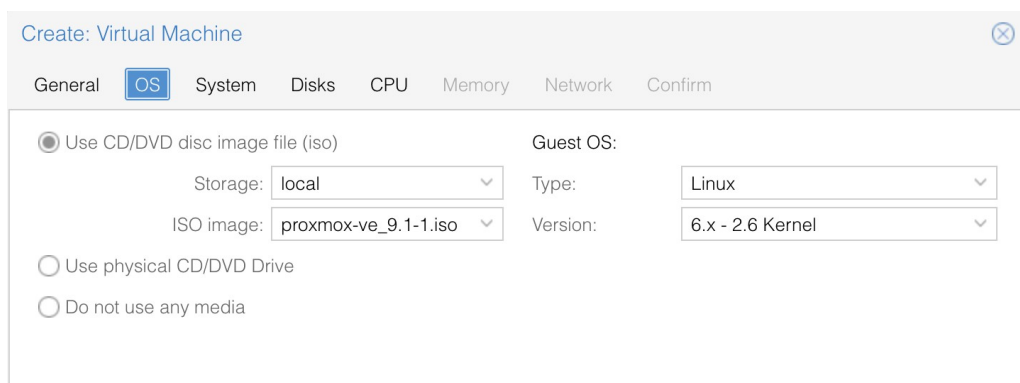
You start by giving a name to your new virtual machine. You should use proxmox1

Note that every VM needs a unique ID, you can leave the default value here.

You don’t need to specify a pool, as for now you have only one physical server.

You then have to specify what OS you want to install.

You will have to specify where (in which storage) to look for, and what ISO image to use (the Proxmox 9.1 ISO you downloaded before). You also have to specify the type of guest operating system (here, choose Linux 6.x – 2.6 kernel).

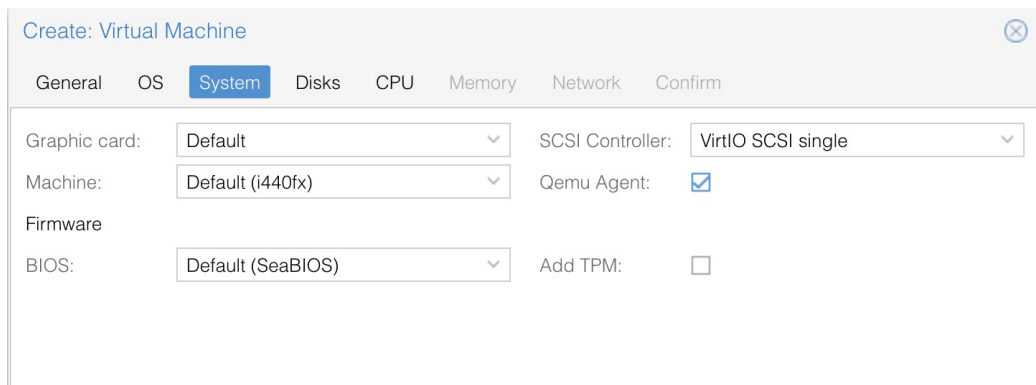


The screenshot shows the 'Create: Virtual Machine' window with the 'OS' tab active. Under the 'Use CD/DVD disc image file (iso)' option, the 'Storage' dropdown is set to 'local' and the 'ISO image' dropdown is set to 'proxmox-ve\_9.1-1.iso'. In the 'Guest OS' section, the 'Type' dropdown is set to 'Linux' and the 'Version' dropdown is set to '6.x - 2.6 Kernel'. Other options like 'Use physical CD/DVD Drive' and 'Do not use any media' are unselected.

You now have to specify a few information about the simulated computer. We will use paravirtualization when possible, in order to improve the performances.

So we choose the VirtIO SCSI controller (for the virtual hard drives), and we also indicate we want to use the Qemu agent (by clicking the checkbox).

To learn more about Qemu agent, look at <https://pve.proxmox.com/wiki/Qemu-guest-agent>



Create: Virtual Machine

General OS **System** Disks CPU Memory Network Confirm

Graphic card: Default

Machine: Default (i440fx)

Firmware

BIOS: Default (SeaBIOS)

SCSI Controller: VirtIO SCSI single

Qemu Agent: ☒

Add TPM: ☐

What is Qemu agent used for ? Do you only have to click on the checkbox for Qemu agent to work ?:

[Answer here](#)

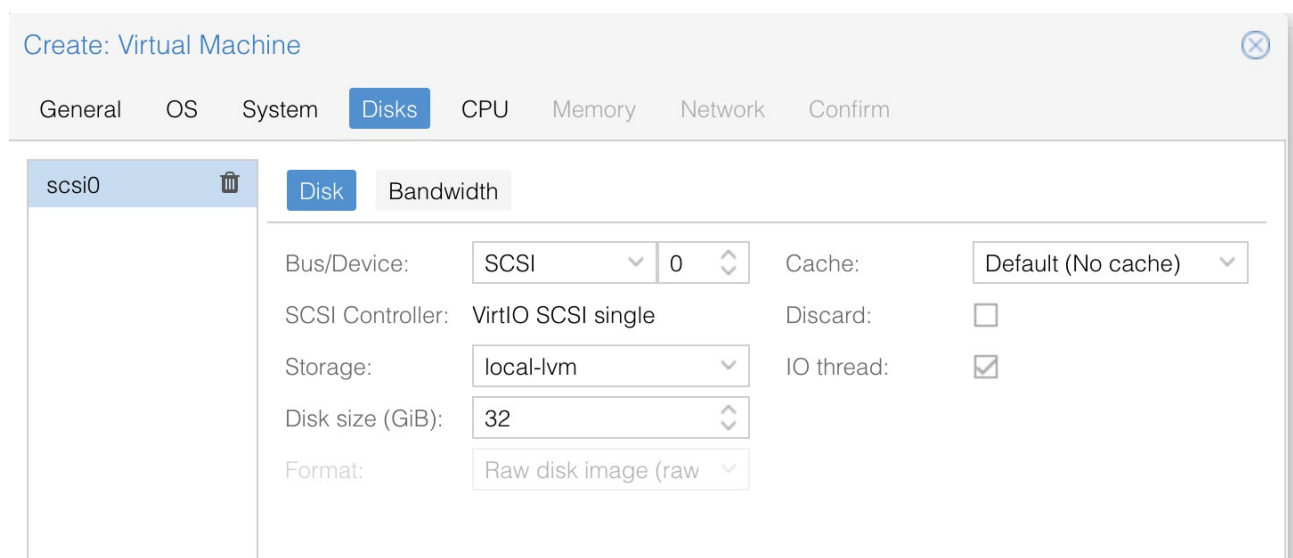
The qemu-guest-agent is used to exchange information between the host and guest, and to execute command in the guest. No, making the Qemu agent work requires two steps. First is to enable it by clicking on the checkbox. Second is to manually install the agent software inside the operating system of your VM.

The next step is setting up the hard drive. A virtual machine can have multiple hard drive, but at creation time, we only specify the characteristics of the one that will be used to install the operating system. It will be possible to add more disks (or other resources like network interfaces) later.

As before, we choose a paravirtualized driver, that will improve the performances. Be careful to select the right storage for your virtual disk.

You can let the size at 32GB, as the space effectively used will only depend on what we install inside this disk. It will not really take 32GB, but more likely only a few GB.

Because of that, it is possible to do over-provizionning , meaning the total size allocated may be well over the physical space available. This is often done in real datacenters, as most users don't use all the storage space allocated to them.



Create: Virtual Machine

General OS System **Disks** CPU Memory Network Confirm

scsi0

Disk Bandwidth

Bus/Device: SCSI 0

Cache: Default (No cache)

SCSI Controller: VirtIO SCSI single

Discard: ☐

Storage: local-lvm

IO thread: ☒

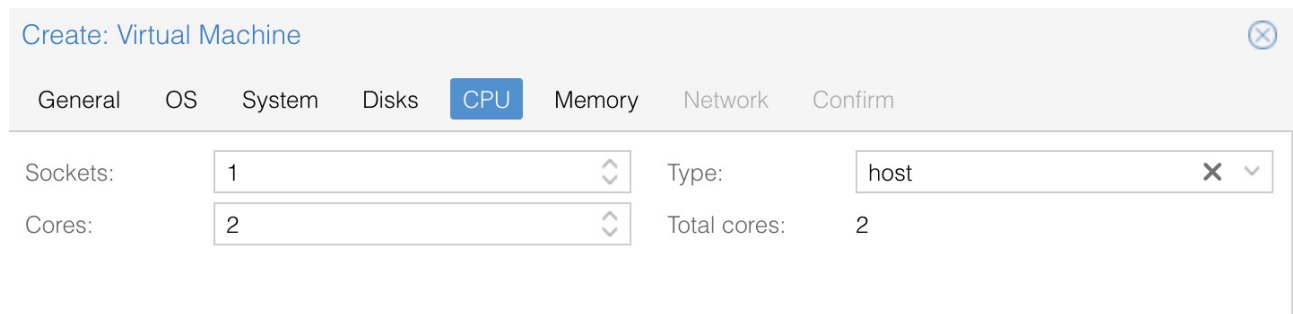
Disk size (GiB): 32

Format: Raw disk image (raw)

For the CPU, usually when it comes to server virtualization, the more cores, the better ... But you have to take into account you will later have to create more instances of your proxmox virtual server (that will also require CPU cores).

Because the CPU of the physical server has 6 cores, I recommend attribution 2 of them to this first VM.

**You have to change to CPU type to “host”.** If you don’t, the virtualization instructions of the physical CPU will not be exposed inside your virtual server, and nested KVM virtual machine you will create inside for your applications will be extremely slow.



The screenshot shows the 'Create: Virtual Machine' window in Proxmox VE, with the 'CPU' tab selected. The window has a title bar with a close button. Below the title bar is a tabbed interface with tabs for 'General', 'OS', 'System', 'Disks', 'CPU', 'Memory', 'Network', and 'Confirm'. The 'CPU' tab is active. It contains four fields: 'Sockets' with a value of '1', 'Cores' with a value of '2', 'Type' with a dropdown menu showing 'host', and 'Total cores' with a value of '2'. The 'Sockets' and 'Cores' fields have up/down arrows. The 'Type' field has a close button and a dropdown arrow. The 'Total cores' field is a simple text input.

| Field       | Value |
|-------------|-------|
| Sockets     | 1     |
| Cores       | 2     |
| Type        | host  |
| Total cores | 2     |

RAM is a critical resource, as Virtual Machines usually use a lot of it. 8GB are available on the provided server, so we will allocate 2.5GB per proxmox server, but also activate ballooning.

To activate ballooning, you may need to click the “advanced” checkbox at the bottom-right of the window.

Set the minimum at 1024 MB and the maximum to 2500 GB.

The screenshot shows a window titled "Create: Virtual Machine" with a close button in the top right corner. Below the title bar is a tabbed interface with tabs for "General", "OS", "System", "Disks", "CPU", "Memory" (which is selected and highlighted in blue), "Network", and "Confirm". The "Memory" tab contains the following settings:

- "Memory (MiB):" with a text input field containing "2500" and a dropdown arrow.
- "Minimum memory (MiB):" with a text input field containing "1024" and a dropdown arrow.
- "Shares:" with a text input field containing "Default (1000)" and a dropdown arrow.
- "Ballooning Device:" with a checked checkbox.
- "Allow KSM:" with a checked checkbox.

At the bottom of the window, there is a "Help" button with a question mark icon, an "Advanced" checkbox which is checked, and "Back" and "Next" buttons.

What is ballooning ? (also explain how it works) :

[Answer here](#)

Ballooning is a term which basically is just, dynamically adjusting the memory so that it only uses the memory it actually needs for the task at hand, this is to prevent using every memory at all time.

If the host runs low on memory, then the balloon drive gives memory from the VM to the host

Next comes the network configuration. In a real setup, we would use multiple network interfaces, dedicated to different tasks (storage network, user network, synchronization between servers network, ...). But for now, we only specify the very first interface, that will allow the virtual machine to communicate with the outside world and internet.

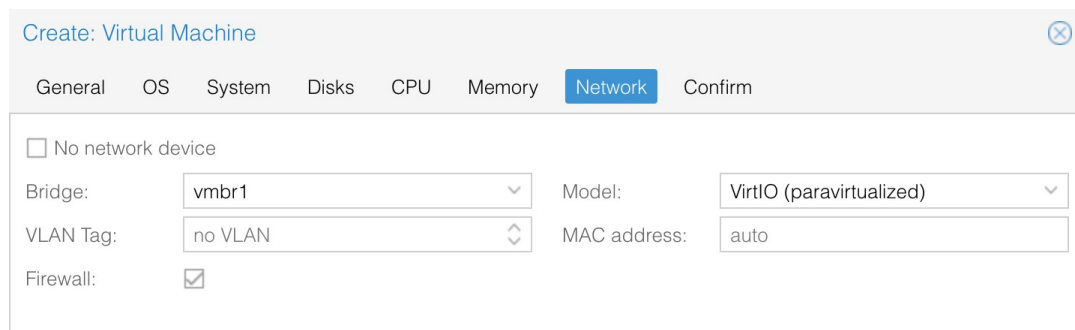
The physical computer (706vdi-iot-xxx) has 5 interfaces (nico, vmbr0, vmbr1, vmbr2 and vmbr3). The first one (nic0) represents the real (physical) network interface of your server and the virtual bridge vmbr0 is directly linked to it. It has an address in the range 10.10.10.xxx. **You should not use vmbr0**, as you would directly send packets and interact with computers on the university's network. The three others (**vmbr1**, **vmbr2** and **vmbr3**) are virtual interfaces to which you will connect your virtual machines.

You are now creating a virtual network interface for your “proxmox1” server. For this interface to be able to communicate, you have to connect it to one of the bridges of your “base” (706vdi-iot.xxx) server.

You will thus connect the network interface of proxmox1 to **vmbr1** of the base server, because this one acts as a bridge to the outside .

Note that this interface uses a NAT. But it will still allow you to connect to the outside.

One again, we use paravirtualization.



The screenshot shows the 'Create: Virtual Machine' dialog box with the 'Network' tab selected. The 'General' tab is also visible. The 'Network' tab contains the following fields:

- ☐ No network device
- Bridge:
- Model:
- VLAN Tag:
- MAC address:
- Firewall: ☒

Please note that for now, we don't specify an IP address. This will be done when we will install the operating system **inside** the virtual machine.

Many other network configurations are possible, but in the case of server virtualization, bridge mode is the most common as it normally allows each VM to have its own IP address and to be easily accessible from the internet.

In our particular case, the network configuration is a little bit different. There is a bridge, but with NAT activated. This is done to prevent your virtual machines to be easily accessed from the outside (even from the rest of the University's network).

We finally get to the confirmation screen where we can verify one last time the informations before committing.

Create: Virtual Machine

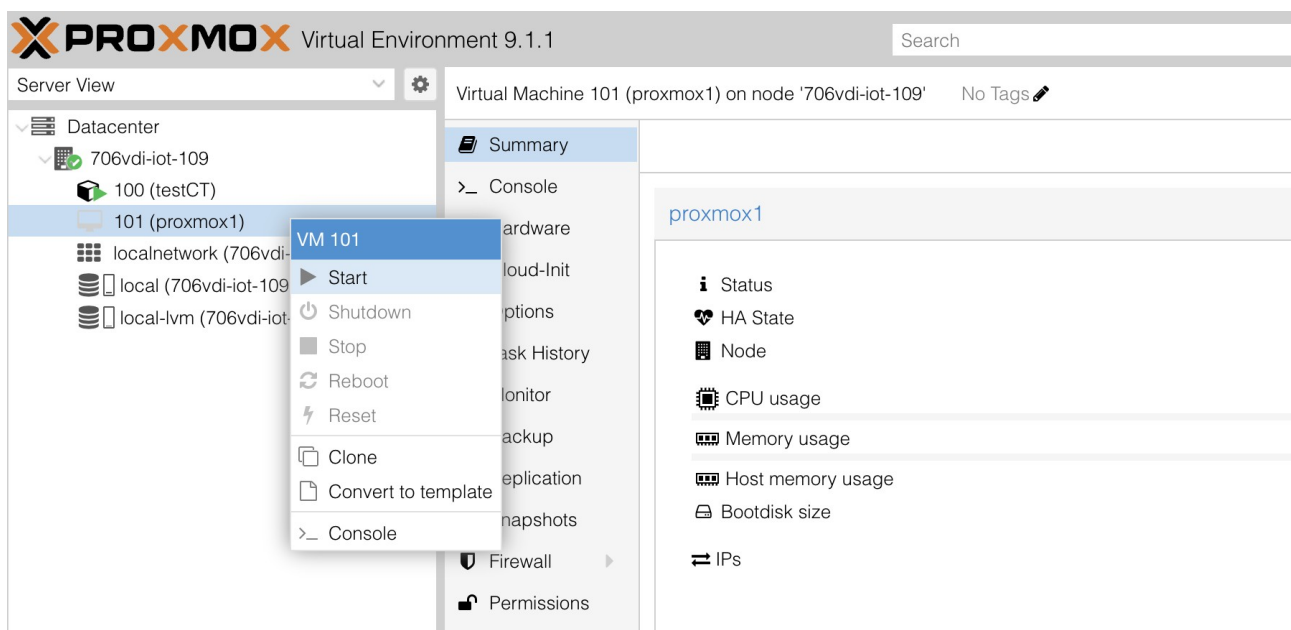
General
OS
System
Disks
CPU
Memory
Network
Confirm

| Key ↑    | Value                                      |
|----------|--------------------------------------------|
| agent    | 1                                          |
| balloon  | 1024                                       |
| cores    | 2                                          |
| cpu      | host                                       |
| ide2     | local:iso/proxmox-ve_9.1-1.iso,media=cdrom |
| memory   | 2500                                       |
| name     | proxmox1                                   |
| net0     | virtio,bridge=vbr1,firewall=1              |
| nodename | 706vdi-iot-109                             |
| numa     | 0                                          |
| ostype   | l26                                        |
| scsi0    | local-lvm:32,iothread=on                   |
| scsihw   | virtio-scsi-single                         |
| sockets  | 1                                          |

☐ Start after created

## Starting the VM and installing Proxmox inside.

Later we will add more virtual devices (network interface and disk), but for now you can now start your VM (right click on the VM, or menu at the top-right of the screen).



You also have to display the console of this VM (in the same menu). You can either display the console on the right panel, or inside a new browser window (recommended for readability reasons). When using new window, you may have to allow your browser to open them.

You should see the boot screen of the Proxmox installation ISO.



Start the installation.

You will have to:

- Accept the licence
- Select the hard-drive to install to (the 32GB virtual disk will do fine)
- Select the country, language and time zone
- Select a password and email for the **root** account. Email is of no importance. There is no point in choosing a strong password here. Use **tpiot2026** as the password. Do not chose a password by yourself. For help or evaluation purpose I may need to connect to your VMs.

After that, you will arrive to the network configuration screen.

There is no DHCP server on this network (and in fact, when you create new servers, you usually want to assign them static IP addresses, to allow users to connect to them easily).

You have to use an IP address in the range 192.168.57.1 to 192.168.57.254, as it is the range used by the vmbr2 virtual interface on the base (706vdi-iot-xxx) server.

As you are on a private network that belongs to you, you could use any address in this range as you like. I recommend using 192.168.57.11

The gateway to the outside world is 192.168.57.1

You should use 194.57.91.200 as DNS server.

Management Interface ● nic0 - bc:24:11:2b:1c:b4 (virtio\_net) ▼

Hostname (FQDN)

IP Address (CIDR)  /

Gateway

DNS Server

☒ Pin network interface names Options

After a summary screen, the installation should take a few minutes. Your virtual machine should then reboot automatically and give you a text login screen:

```

-----
Welcome to the Proxmox Virtual Environment. Please use your web browser to
configure this server - connect to:

  https://192.168.57.11:8006/

-----

proxmox1 login:

```

You don't need to log here, as the management is normally done through the web server. Normally you would have to open the given url in a web browser: <https://192.168.100.11:8006> Except that this IP address is on a private network, and you can't access it directly.

To access your server from the classroom, you will need to configure a port forwarding on your server (706vdi-iot-xxx). This can be done with the following command:

```
root@706vdi-iot-xxx:~$ sudo iptables -t nat -A PREROUTING -i vmbro0 -p tcp --dport 8007 -j DNAT --to-destination 192.168.57.11:8006
```

note:

- -d specify the address of your base server (706vdi-iot-xxx)
- --dport has to be different from 8006
- --to-destination specify the IP address of proxmox1

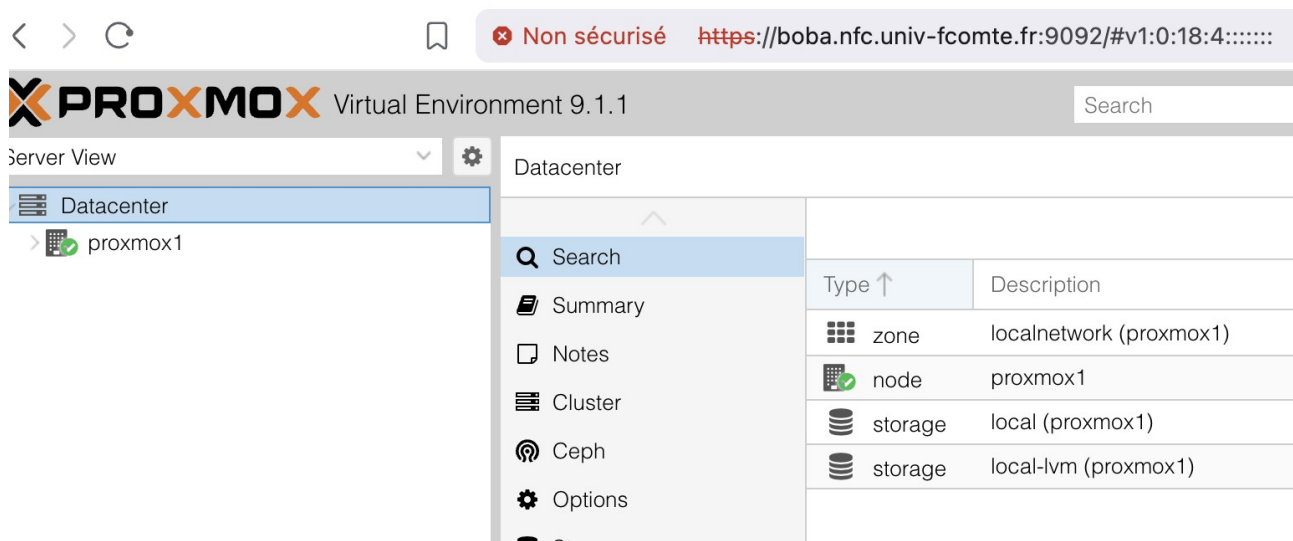
also note:

There is a double port forwarding here

boba:9092 → 706vdi-iot-109:8007 → proxmox1:8006

After this, you can connect directly to proxmox1 at the address:

<https://boba.nfc.univ-fcomte.fr:9092>



On this server, configure the apt sources as you did on 706vdi-iot-xxx

By doing so, you will test its internet connection.